DESIGN REPORT

VETERINARY CLINIC APPOINTMENT MANAGEMENT SYSTEM

SESEIU24001 - Võ Châu Anh

---

1. Assumptions

These are the assumptions I used:

- Currency is VND, stored as `Double` and formatted by a `vnd` helper.
- `Appointment.id`, `pet`, `createdDate` are immutable (`let`) - identity and owning pet never change after the booking is created.
- `AppointmentStatus` has a strict lifecycle: `booked` → `confirmed` → `checkedIn` → `completed`, with `cancelled` as a terminal branch, and a controlled amendment path `completed` → `confirmed` available only to the Manager with a written reason.
- Charges lock as soon as the appointment is confirmed (or checked in / completed). After that, nobody can change `baseFee` or `promotion`,not even after an amendment. Extra services can still be added until the visit is completed or cancelled.
- Emergency appointments auto-confirm at creation (`initialStatus: .confirmed`) because they receive immediate attention. They also skip the same-day check-in rule, and cannot be rescheduled.
- Follow-up consultation: `RoutineConsultation` gets a 50 % consultation-fee discount if the previous visit is within 14 days before the booking's createdDate (measured at booking time, not at check-in).
- Vaccination requires: vaccine selected, species match, and booster interval respected against the pet's vaccination history. Completing a vaccination appends a `VaccinationRecord` to the pet; reopening removes the last matching record.
- Four roles: `owner` (view + request change only), `receptionist` (edit booking + add services), `vet` (add services + complete visit), `manager` (everything except request change, including fee changes and amendments).
- Change requests (cancel, reschedule) are submitted by the owner and resolved by receptionist/manager. Only one pending request at a time.

- Promotions are pre-seeded (`Promotion.samples`) with a scope - either `total` (applies to the whole subtotal) or `servicesOnly` (extras only). They have an eligibility set per appointment type and an expiry date.

- No backend - `ClinicManager(sample: true)` loads eight appointments with varied statuses; `ClinicManager.preview()` reuses it for SwiftUI previews.

2. Type identification

Classes:

Class	Responsibility
`Appointment`	Base class. Holds identity (`id`, `pet`, `createdDate`), shared state (`scheduledDate`, `status`, `notes`, `baseFee`, `promotion`, `services`, `pendingRequest`, `amendments`, `completedDate`). Owns the entire lifecycle: confirm, check-in, complete, cancel, reschedule, add service, change fee, apply/remove promotion, reopen completed visit, submit and resolve change requests, plus the shared `authorize` / `ensureNotClosed` / `ensureEditable` guards.
`RoutineConsultation`	Standard or follow-up consultation. Adds `previousVisitDate` and derived `isFollowUp` / `followUpDiscountApplies`. Overrides `chargeLines()` to apply the 50 % follow-up discount.
`VaccinationAppointment`	Vaccination visit. Adds `vaccine: Vaccine?` with `selectVaccine`, `validateVaccine`, and lifecycle hooks `validateReady`, `didComplete`, `didReopen` to record/remove the pet's vaccination history.
`EmergencyAppointment`	Emergency visit. Adds `urgency: UrgencyLevel`, `symptoms`, and computed `isAfterHours`. Auto-confirmed at creation. Adds urgency + after-hours surcharges to the fee.
`Owner`	Pet owner identity (id, name, phone).
`Pet`	Pet identity (id, name, species, age, owner) and `vaccinationHistory`.
`ClinicManager`	`ObservableObject`. Owns `[Appointment]` and `promotions`, provides search (sorted by priority + scheduledDate), add, totals (`totalEstimated`, `lockedCount`), and the `attempt(_:)` wrapper that turns thrown errors into visible messages.

Structs:

Struct	Fields	Purpose
VaccinationRecord	let id = UUID(), vaccineName, date	One historical vaccination entry.
Vaccine	id (from code), code, name, price, boosterIntervalDays, species: Set<Species>; static catalog	Vaccine catalogue with pricing and eligibility.
ServiceItem	id (from code), code, name, price; static catalog	Extra service catalogue.
Promotion	id (from code), code, title, percentOff, scope: DiscountScope, eligible: Set<AppointmentType>, validUntil; methods isValid(on:), isEligible(for:); static samples	Discount configuration.
ChangeRequest	let id = UUID(), kind: ChangeRequestKind, date	One pending owner request.
Amendment	let id = UUID(), reason, date	One amendment log entry.
InfoItem	let id = UUID(), label, value	Generic display row.
ChargeLine	let id = UUID(), label, amount	One row on a bill.
ChargeBreakdown	lines, promotion, subtotal, discount, total	Result of the shared breakdown property , recalculated every time it is read.

Enums:
AppointmentType (routine/vaccination/emergency), AppointmentStatus (booked/confirmed/checkedIn/completed/cancelled), Species (dog/cat/rabbit, with icon), UrgencyLevel (urgent/critical, with surcharge), DiscountScope (total/servicesOnly), ChangeRequestKind (cancel/reschedule with title), Permission (6 cases), UserRole (owner/receptionist/vet/manager, with can(_:)). VetError covers every validation path currently used.

3. Properties & Methods

Shared stored properties in `Appointment`:

- `let id`, `let pet`, `let createdDate` - immutable
- `private(set) var scheduledDate`, `status`, `notes`, `baseFee`, `promotion`, `services: [ServiceItem]`, `pendingRequest`, `amendments: [Amendment]`, `completedDate: Date?`

Overridable policy hooks in `Appointment`:

- `type`, `typeName`, `icon`, `feeLabel`, `priority`, `allowedCheckInStatuses`, `requiresSameDayCheckIn`, `allowsReschedule`, `typeSpecificActions`, `typeSpecificRows`, `searchableText`
- `chargeLines() -> [ChargeLine]` - main polymorphic fee calculator
- `validateReady()` - additional precondition before `confirm` / `checkIn` (empty in base)
- `didComplete()`, `didReopen()` - lifecycle side-effects (empty in base)

Shared `final` computed members:

- `chargesLocked` (`status == .confirmed || .checkedIn || .completed`)
- `isClosed` (`status == .completed || .cancelled`)
- `breakdown: ChargeBreakdown` - sums `chargeLines()` + `services`, applies promotion using its `scope`
- `estimatedCharges` - `0` if cancelled, else `breakdown.total`
- `availableActions: [String]` - status-driven list plus `typeSpecificActions`

Shared `final` methods:

- `authorize(_:_:action:)`, `ensureNotClosed(_:)`, `ensureEditable(_:)`
- `updateNotes`, `confirm`, `checkIn`, `complete`, `cancel`, `reschedule`
- `addService`, `updateBaseFee`, `applyPromotion`, `removePromotion`
- `reopenCompletedVisit(reason:by:)`
- `submitRequest(_:by:)`, `resolveRequest(approve:by:)`

Shared non-final members that are not overridden by any current subclass:

- `activePromotion` (returns `nil` if the stored promotion has expired)
- `detailRows` (identity rows + `typeSpecificRows` + notes)

Subclass-specific stored properties (access level noted):

Class	Property	Access
`RoutineConsultation`	`previousVisitDate: Date?`	`let`
`VaccinationAppointment`	`vaccine: Vaccine?`	`private(set) var`
`EmergencyAppointment`	`urgency: UrgencyLevel`, `symptoms: String`	`let`

Subclass-specific computed / methods:

- RoutineConsultation: isFollowUp, followUpDiscountApplies
- VaccinationAppointment: validateVaccine(_:) (species + booster interval), selectVaccine(_:by:) (authorized + not locked)
- EmergencyAppointment: isAfterHours (hour < 8 or ≥ 18)

Pet / Owner / ClinicManager:

- Pet: let id, species, owner; private(set) var name, ageYears, vaccinationHistory; methods lastVaccination(named:), recordVaccination(_:on:), removeLastVaccination(named:)
- Owner: let id; private(set) var name, phone
- ClinicManager: @Published private(set) var appointments; @Published var currentRole; let promotions = Promotion.samples; methods appointment(withID:), search(_:type:), addAppointment(_:), attempt(_:), totalEstimated, lockedCount

4. Inheritance & Polymorphism

Hierarchy:

```
Appointment
├── RoutineConsultation
├── VaccinationAppointment
└── EmergencyAppointment
```

Appointment is not final (it must be inheritable). The three subclasses are final. Every lifecycle and state-mutating method is final in the base, so subclasses can only override the policy hooks and lifecycle callbacks - this keeps state transitions and validation consistent across types.

Inherited (written once): identity, chargesLocked / isClosed logic, breakdown math (including scope-aware discount), the entire confirm / check-in / complete / cancel / reschedule workflow, note and fee editing, service addition, promotion handling, amendment path, change-request workflow.

Complete override matrix (dash = base default kept):

Member	RoutineConsultation	VaccinationAppointment	EmergencyAppointment
type	.routine	.vaccination	.emergency
typeName	"Routine Consultation" / "Follow-up Visit"	"Vaccination"	"Emergency (Urgent)" / "(Critical)"
icon	stethoscope / arrow.uturn.backward.circle.fill	syringe.fill	cross.case.fill

feeLabel	"Consultation fee"	"Administration fee"	"Emergency fee"
priority	1	2	0 (highest)
allowedCheckInStatuses	-	-	[.booked, .confirmed]
requiresSameDayCheckIn	-	-	false
allowsReschedule	-	-	false
typeSpecificActions	follow-up only: "Review previous care"	"Select vaccine" (only if not locked)	"Immediate triage"
searchableText	-	vaccine name	symptoms
chargeLines()	follow-up 50 %, else full fee	admin fee + vaccine price	emergency fee + urgency surcharge + after-hours 50 %
typeSpecificRows	previous visit + rate	vaccine + booster interval	priority / urgency / symptoms / after-hours
validateReady()	-	vaccine must be selected + valid	-
didComplete()	-	record vaccination on pet	-
didReopen()	-	remove last vaccination from pet	-

detailRows and activePromotion are declared in the base and not overridden - they use typeSpecificRows and the stored promotion directly.

Polymorphism in practice:

- ClinicManager.totalEstimated sums estimatedCharges across all appointments. estimatedCharges uses the shared breakdown, which calls each appointment's chargeLines(). The manager never branches on type.
- AppointmentCard and ChargeCard render breakdown.lines, estimatedCharges and pills without any switch - each subtype contributes its own fee rows.

- `checkIn` polymorphism: the same `checkIn` call succeeds for a confirmed Routine only when the current date is the scheduled day, but for an `EmergencyAppointment` it succeeds from `.booked` as well, and without the same-day requirement because `allowedCheckInStatuses` includes `.booked` and `requiresSameDayCheckIn` is `false`. A single implementation handles all three cases through the hooks.
- Lifecycle hooks: `complete` calls `didComplete()` polymorphically. For a `VaccinationAppointment` that records the vaccination on the pet; for the others it is a no-op. `reopenCompletedVisit` calls `didReopen()` symmetrically, removing the vaccination record.
- `validateReady` polymorphism: `confirm` and `checkIn` both call `validateReady()` before the state transition. The base implementation is empty; `VaccinationAppointment` overrides it to require a valid vaccine - so a vaccination appointment cannot be confirmed or checked in without one, and the rule lives in exactly one place.

When protocol/composition would be better: if future services require cross-cutting traits (e.g., an appointment that is both a vaccination and an emergency), a class tree cannot express it. Protocols such as `Billable`, `Reschedulable`, `SpeciesRestricted`, combined with small value components (a `FeePolicy`, a `BoosterRule`), would then be cleaner. I chose class inheritance here because all current types share one unified state machine (`booked → confirmed → checkedIn → completed` with cancel/reopen) and the differences are entirely within a handful of hooks.

5. Encapsulation & Validation

Access-control decisions:

Data	Access modifier	Reason
`Appointment.id`, `pet`, `createdDate`	`let`	Identity / owning pet / creation timestamp - never change
`Appointment.scheduledDate`, `notes`, `baseFee`, `promotion`, `services`, `pendingRequest`, `amendments`, `completedDate`	`private(set) var`	Each has a well-defined set of `final` mutating methods that authorize and validate first
`Appointment.status`	`private(set) var`	Written only by `confirm`, `checkIn`, `complete`, `cancel`, `reopenCompletedVisit`

RoutineConsultation.previousVisitDate	let	Set at booking time. The 14-day follow-up check compares it with createdDate (the booking date).
VaccinationAppointment.vaccine	private(set) var	Only selectVaccine(_:by:) may write, and only before charges lock
EmergencyAppointment.urgency, symptoms	let	Emergency triage fields fixed at creation
Pet.id, species, owner	let	Identity / species / ownership - never change
Pet.name, ageYears	private(set) var	Readable everywhere; no writer currently exists (effectively read-only after init)
Pet.vaccinationHistory	private(set) var	Only recordVaccination / removeLastVaccination may write - driven by appointment lifecycle
Owner.id	let	Immutable
Owner.name, phone	private(set) var	No external writer currently
ClinicManager.appointments	@Published private(set)	Views read; only the manager appends
ClinicManager.currentRole	@Published (writable)	Selected by the user
ClinicManager.promotions	let = Promotion.samples	Seed catalogue; not mutated

Where each private(set) field is written - the single gate per field:

- status ← confirm, checkIn, complete, cancel, reopenCompletedVisit (5 final methods, each authorized)
- scheduledDate ← reschedule only
- notes ← updateNotes only

- baseFee ← updateBaseFee only (blocked once chargesLocked)
- promotion ← applyPromotion, removePromotion (blocked once chargesLocked)
- services ← addService only (blocked once closed)
- pendingRequest ← submitRequest (set), resolveRequest / cancel (clear)
- amendments ← reopenCompletedVisit only
- completedDate ← complete (set), reopenCompletedVisit (clear)

Shared guards: authorize and ensureNotClosed / ensureEditable are the entry conditions every operation passes through, so cancelled, completed and in-progress appointments are all rejected in one place.

Validation rules:

Rule	Where	Error
Appointment ID not empty	addAppointment	emptyField
Amendment reason not empty	reopenCompletedVisit	reasonRequired
No duplicate appointment ID	addAppointment	duplicateAppointment
Role must have the required permission	authorize (all mutating methods)	unauthorized
Closed appointments cannot be modified	updateNotes, confirm, addService, updateBaseFee, applyPromotion, removePromotion	appointmentCancelled / completedVisitLocked
In-progress visit cannot be cancelled or rescheduled	cancel, reschedule (via ensureEditable)	visitInProgress
confirm requires status == booked	confirm	invalidStatus
checkIn requires status in allowedCheckInStatuses	checkIn	notConfirmed
checkIn requires same-day unless overridden	checkIn (only non-emergency)	notScheduledToday

checkIn from cancelled / completed / already-checked-in	checkIn	appointmentCancelled / completedVisitLocked / invalidStatus
complete requires status == checkedIn	complete	notCheckedIn
reschedule requires allowsReschedule	reschedule, submitRequest	rescheduleNotAllowed
New date must be in the future	reschedule, submitRequest	dateInPast
Fee finite and > 0	updateBaseFee	invalidAmount
Fee / promotion changes blocked after confirm	updateBaseFee, applyPromotion, removePromotion	chargesLocked
Promotion valid + eligible + not already applied	applyPromotion	promotionExpired / promotionNotEligible / promotionAlreadyApplied
Service not already added	addService	serviceAlreadyAdded
Reopen requires status == completed + reason not empty	reopenCompletedVisit	invalidStatus / reasonRequired
Submit request: no pending request already	submitRequest	requestPending
Resolve request: requires one to exist	resolveRequest	noPendingRequest
Vaccine species matches pet	validateVaccine	vaccineSpeciesMismatch
Booster interval respected	validateVaccine	vaccineTooSoon
Vaccine selected before confirm / check-in	VaccinationAppointment.validateReady	vaccineNotSelected
Vaccine not changed after charges lock	selectVaccine	chargesLocked

6. SwiftUI implementation

Screen structure:

- `ContentView` - `TabView` with Appointments and Charges. Injects `ClinicManager(sample: true)` as `@StateObject`.
- `AppointmentsView` - search field, segmented filter by `AppointmentType`, `RoleBanner`, and a list of `AppointmentCard`. Emergency rows are tinted for visual priority. Cards show pet + owner, ID, type, scheduled time, `estimatedCharges`, "Charges locked" pill, "PRIORITY" pill for emergency.
- `ChargesView` - banner with `totalEstimated` and locked/total count, then a list of `ChargeCard` (breakdown lines + promotion + total; cancelled shows "CANCELLED").
- `AppointmentDetailView` - one layout for all types:
    - Header (icon, pet · owner, ID + status pill, acting-role row)
    - Visit details - `detailRows` (which already includes `typeSpecificRows`) + pet / owner phone + edit-notes button
    - Estimated charges - `breakdown.lines` + promotion row + `estimatedCharges` + locked pill
    - Available actions - `availableActions` (status- and type-driven)
    - Type-specific sections (`VaccinationSection` / `EmergencySection`) - only the cast differs
    - `WorkflowSection` (confirm, check-in, complete, cancel)
    - `ScheduleSection` (reschedule, owner requests, amendment)
    - `ServicesSection` (add service)
    - `PricingSection` (change fee, apply/remove promotion)
- `EditNotesView` - sheet for notes only. ID and pet are shown as read-only `LabeledContent`, matching `let` immutability.

Where polymorphic behavior is visible to the user:

- Emergency card shows a "PRIORITY" pill and uses red tint on the row; Routine and Vaccination do not.
- Fee breakdowns differ: Routine shows one line (or a "50 % follow-up" line); Vaccination shows admin + vaccine; Emergency shows emergency fee + urgency surcharge (+ after-hours).
- Pet's vaccination history appears only on `VaccinationSection`; emergency triage info appears only on `EmergencySection`.
- `availableActions` updates per status and per type, showing e.g. "Immediate triage" only for emergencies, and "Review previous care" only for follow-ups.

7. Conclusion

The design keeps shared lifecycle and validation in `Appointment` - identity, charge locking, breakdown math, the confirm → check-in → complete → cancel chain, the amendment procedure, and the change-request workflow - and pushes only

type-specific variance into subclasses through a small set of overridable hooks: `type`, `typeName`, `icon`, `feeLabel`, `priority`, `allowedCheckInStatuses`, `requiresSameDayCheckIn`, `allowsReschedule`, `typeSpecificActions`, `searchableText`, `chargeLines()`, `typeSpecificRows`, `validateReady()`, `didComplete()`, and `didReopen()`. Sensitive state (`id`, `pet`, `createdDate`, `status`, `scheduledDate`, `baseFee`, `promotion`, `services`, `amendments`, `completedDate`, `vaccine`, `urgency`, `symptoms`, `vaccinationHistory`) is `let` or `private(set)` with a clearly identified single-gate mutator per field, and every one of those mutators is `final` and starts with `authorize(...)` then validation. The one exception is `Pet.vaccinationHistory`, which only changes through `didComplete` / `didReopen` of a vaccination appointment. Completed visits are protected by the amendment-only path; cancelled visits and locked charges are rejected through `VetError`. The result is a compact, extensible model that scales by adding a new subclass with a handful of overrides, and that is straightforward to demonstrate in SwiftUI with clear valid and rejected feedback.